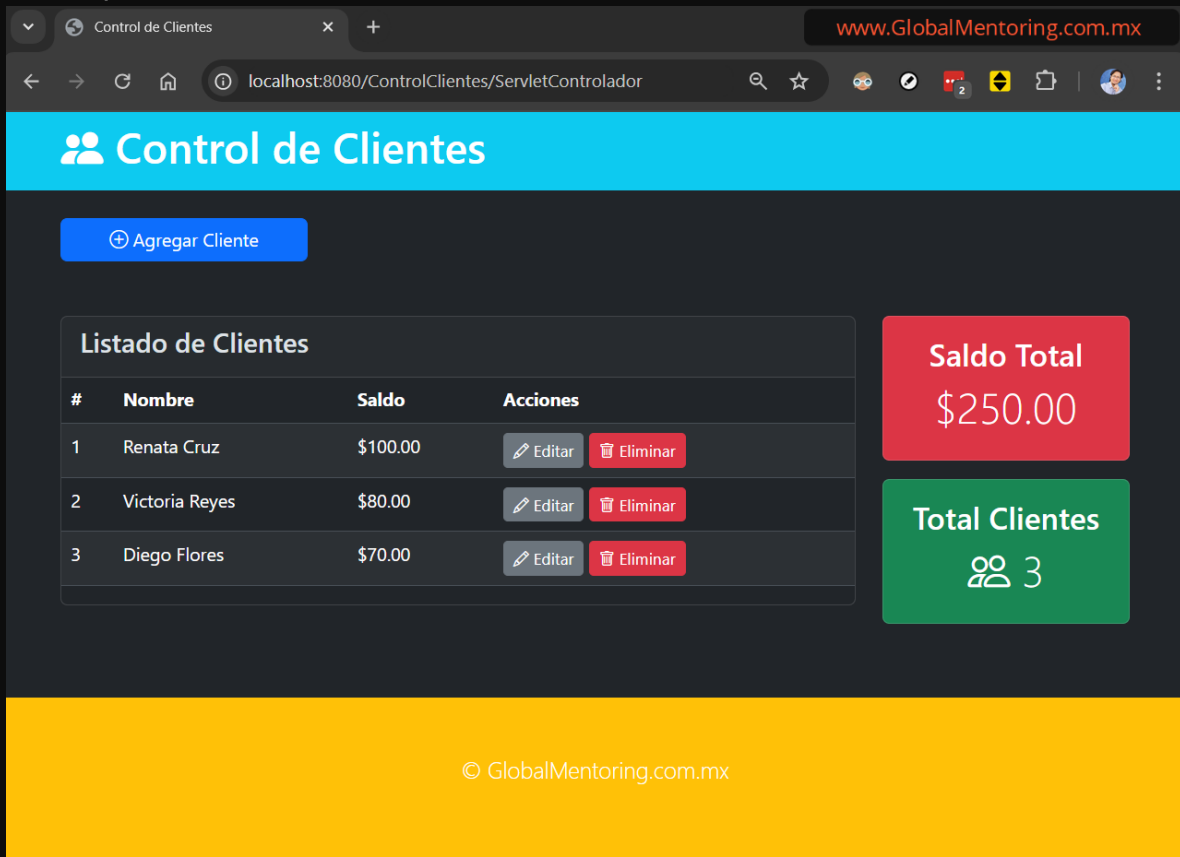


Proyecto Control de Clientes



Guía para crear el Proyecto de Control de Clientes con el patrón de diseño MVC, Servlets, JSPs, Bootstrap y MySQL

Este documento te guiará en la creación de una aplicación web en Java utilizando NetBeans y el servidor GlassFish con el soporte de Jakarta EE. La aplicación permite gestionar una lista de clientes, mostrando un listado de clientes, agregando, editando y eliminando registros. Utilizaremos las mejores prácticas de desarrollo aplicadas a las últimas versiones de cada tecnología.

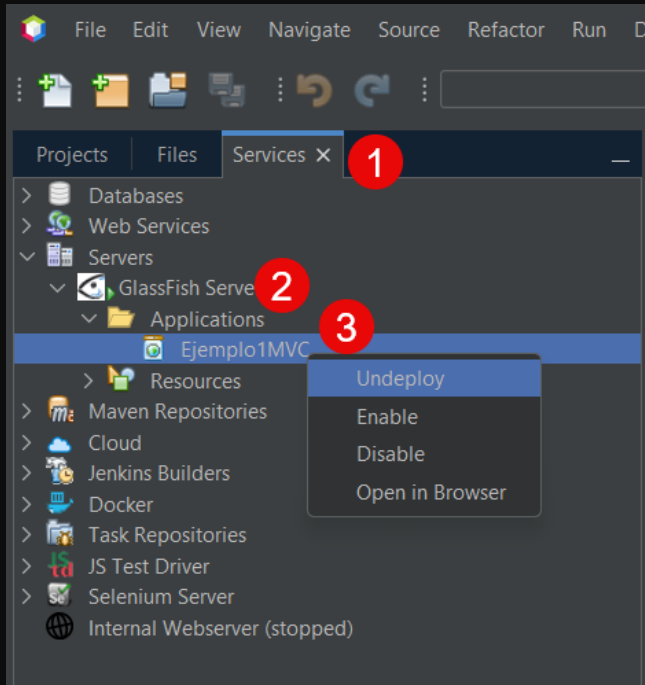
Configuración del Proyecto

1. **Crear el Proyecto en NetBeans:**
 - Abre NetBeans.
 - Selecciona **Archivo > Nuevo Proyecto > Java con Maven > Aplicación Web**.
 - Asigna el nombre **ControlClientes** al proyecto.
 - Escoge el servidor GlassFish y selecciona Jakarta EE como el framework principal.
2. **Configuración de Dependencias en `pom.xml`:**

- Asegúrate de agregar las dependencias para MySQL, Jakarta EE y Apache Commons DBCP2 para el manejo del pool de conexiones. Si ya están incluidas, verifica que estén correctamente configuradas.

Recomendación antes de continuar:

Se recomienda hacer undeploy de las aplicaciones desplegadas en Glassfish para comenzar de cero con Glassfish. Hacer esto con todas las aplicaciones que ya no estés utilizando para limpiar el servidor de Glassfish y enfocarnos en este proyecto final.



Base de Datos y Conexión

1. Configuración de la Base de Datos llamada control_clientes:

- Utiliza MySQL con la siguiente estructura de tabla para el proyecto:

```
CREATE TABLE `clientes` (  
  `id_cliente` int(11) NOT NULL AUTO_INCREMENT,  
  `nombre` varchar(45) DEFAULT NULL,  
  `apellido` varchar(45) DEFAULT NULL,  
  `email` varchar(45) DEFAULT NULL,  
  `telefono` varchar(45) DEFAULT NULL,  
  `saldo` double DEFAULT NULL,  
  PRIMARY KEY (`id_cliente`)  
);
```

- Datos de ejemplo:

<https://www.globalmentoring.com.mx>

```
INSERT INTO `clientes` VALUES ('Renata', 'Cruz', 'rcruz@mail.com', '5566778899', 100),
                                ('Victoria', 'Reyes', 'vreyes@mail.com', '5511223344', 80),
                                ('Diego', 'Flores', 'dflores@mail.com', '5544332211', 70);
```

Pasos para Agregar el Driver de MySQL a GlassFish

1. Descargar el Driver de MySQL:

- Descarga el archivo JAR del conector MySQL (por ejemplo, `mysql-connector-java-8.x.x.jar`) desde la [página oficial de MySQL](#).
- También pueden descargar el archivo .jar de los recursos de este video o del siguiente link:

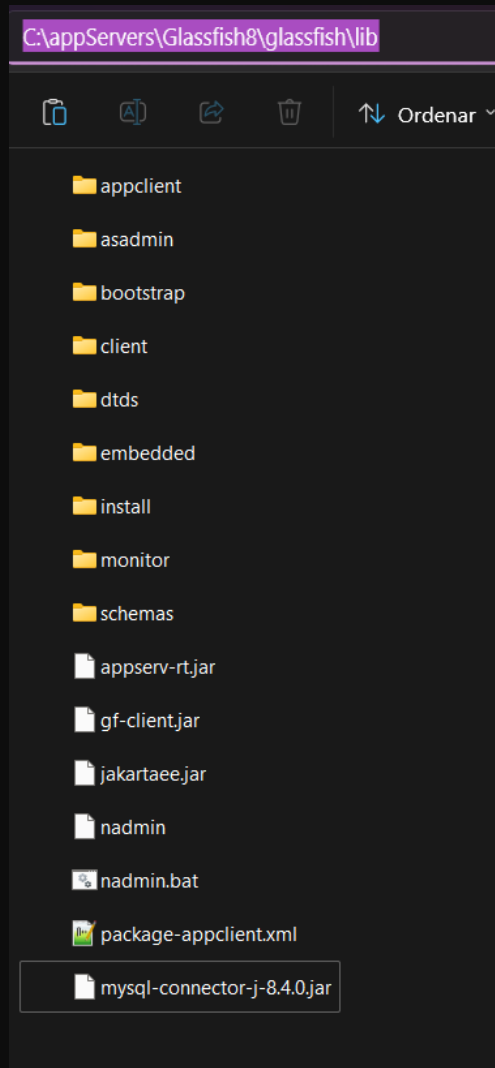
<https://repo1.maven.org/maven2/com/mysql/mysql-connector-j/8.4.0/mysql-connector-j-8.4.0.jar>

2. Copiar el JAR en el Directorio lib de GlassFish:

- Coloca el archivo `mysql-connector-java-8.x.x.jar` en la carpeta `lib` de GlassFish:

`C:\appServers\Glassfish8\glassfish\lib`

- Este es el directorio donde GlassFish busca los drivers de base de datos al iniciarse.



3. Reiniciar GlassFish:

- Después de agregar el archivo JAR, reinicia el servidor GlassFish para que cargue el nuevo driver.

Agregar la dependencia BasicDataSource a Maven

La clase `BasicDataSource` se encuentra en la biblioteca **Apache Commons DBCP** (Database Connection Pooling), que es parte del proyecto Apache Commons. Esta biblioteca permite gestionar un pool de conexiones de base de datos en Java, facilitando una gestión eficiente de conexiones.

Para agregar la dependencia de **Apache Commons DBCP** en tu proyecto Maven (`pom.xml`), puedes incluir la siguiente configuración:

```
<dependency>
  <groupId>org.apache.commons</groupId>
```

```
<artifactId>commons-dbcp2</artifactId>
<version>2.12.0</version>
</dependency>
```

- **org.apache.commons:** Este es el grupo para las bibliotecas de Apache Commons.
- **commons-dbcp2:** Es el nombre del artefacto que contiene `BasicDataSource` y otras clases de conexión.
- Después de agregar esta dependencia a `pom.xml`, puedes usar `BasicDataSource` para crear y gestionar un pool de conexiones en tu proyecto.

Agregamos la dependencia de MySQL a Maven

```
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.4.0</version>
</dependency>
```

Creación del Modelo Cliente

1. Clase Cliente (`Cliente.java`):

- **Descripción:** Esta clase representa la entidad `Cliente` y su estructura en la base de datos.
- Creamos un paquete llamado **modelo** y agregamos la clase **Cliente** con el siguiente código.

```
package modelo;

/**
 *
 * @author ubaldo
 */
public class Cliente {
    private int idCliente;
    private String nombre;
    private String apellido;
    private String email;
    private String telefono;
    private double saldo;

    public Cliente() {
    }

    public Cliente(int idCliente) {
        this.idCliente = idCliente;
    }
}
```

```
}

    public Cliente(String nombre, String apellido, String email, String telefono, double
saldo) {
        this.nombre = nombre;
        this.apellido = apellido;
        this.email = email;
        this.telefono = telefono;
        this.saldo = saldo;
    }

    public Cliente(int idCliente, String nombre, String apellido, String email, String
telefono, double saldo) {
        this.idCliente = idCliente;
        this.nombre = nombre;
        this.apellido = apellido;
        this.email = email;
        this.telefono = telefono;
        this.saldo = saldo;
    }

    public int getIdCliente() {
        return idCliente;
    }

    public void setIdCliente(int idCliente) {
        this.idCliente = idCliente;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellido() {
        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
    }
}
```

```
public String getEmail() {
    return email;
}

public void setEmail(String email) {
    this.email = email;
}

public String getTelefono() {
    return telefono;
}

public void setTelefono(String telefono) {
    this.telefono = telefono;
}

public double getSaldo() {
    return saldo;
}

public void setSaldo(double saldo) {
    this.saldo = saldo;
}

@Override
public String toString() {
    return "Cliente{" + "idCliente=" + idCliente + ", nombre=" + nombre + ",
apellido=" + apellido + ", email=" + email + ", telefono=" + telefono + ", saldo=" +
saldo + '}';
}
}
```

Clase de Conexión (Conexion.java):

- **Descripción:** Esta clase maneja la conexión a la base de datos utilizando un pool de conexiones.

```
package datos;

import java.sql.Connection;
import java.sql.SQLException;
import javax.sql.DataSource;
import org.apache.commons.dbcp2.BasicDataSource;
```

```
/**
 *
 * @author ubaldo
 */
public class Conexion {
    private static final String JDBC_URL =
        "jdbc:mysql://localhost:3306/control_clientes?"
        +
        "useSSL=false&useTimezone=true&serverTimezone=UTC&allowPublicKeyRetrieval=true";
    private static final String JDBC_USER = "root";
    private static final String JDBC_PASSWORD = "admin";
    private static BasicDataSource dataSource;

    public static DataSource getDataSource(){
        if(dataSource == null){
            dataSource = new BasicDataSource();
            dataSource.setUrl(JDBC_URL);
            dataSource.setUsername(JDBC_USER);
            dataSource.setPassword(JDBC_PASSWORD);
            dataSource.setInitialSize(10);
            dataSource.setMaxTotal(50);
        }
        return dataSource;
    }

    public static Connection getConnection() throws SQLException {
        return getDataSource().getConnection();
    }
}
```

- **Mejoras Aplicadas:**

- Configuración de parámetros de `BasicDataSource` para inicializar con un tamaño de pool adecuado y un límite máximo de conexiones.

Explicación de la Clase `Conexion`

1. Constantes de Configuración de la Base de Datos:

- `JDBC_URL`: Especifica la URL de conexión a la base de datos MySQL, incluyendo parámetros como `useSSL=false`, que desactiva SSL; `useTimezone=true`, que asegura que se respeten las zonas horarias; y `allowPublicKeyRetrieval=true`, necesario para ciertos modos de autenticación de MySQL.

- **JDBC_USER y JDBC_PASSWORD:** Son las credenciales de usuario y contraseña para acceder a la base de datos MySQL.
- 2. **Campo dataSource:**
 - Es una instancia de `BasicDataSource` de Apache Commons DBCP. Este objeto gestiona un pool de conexiones, permitiendo que la aplicación reutilice conexiones en lugar de abrir y cerrar una nueva conexión cada vez que se necesita.
- 3. **Método `getDataSource()`:**
 - Este método proporciona acceso al `DataSource`. Si `dataSource` es null (es decir, si aún no se ha inicializado), lo crea y configura.
 - **Configuraciones del pool:**
 - `setInitialSize(10)`: Define que el pool inicie con 10 conexiones abiertas.
 - `setMaxTotal(50)`: Define un límite máximo de 50 conexiones en el pool.
 - Estas configuraciones optimizan el uso de recursos y mejoran el rendimiento de la aplicación, ya que las conexiones a la base de datos se reutilizan.
- 4. **Método `getConnection()`:**
 - Este método obtiene una conexión desde el `DataSource` (pool de conexiones) y la devuelve al solicitante.
 - **Manejo de Excepciones:** Lanza `SQLException` si ocurre algún problema al obtener la conexión. Este enfoque permite que el código que llama a este método maneje los errores de conexión.

DAO para Gestión de Clientes (`ClienteDaoJDBC`)

Descripción: Esta clase maneja las operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre la entidad Cliente en la base de datos, utilizando las mejores prácticas y el soporte de `BasicDataSource` para la conexión.

```
package datos;

import java.sql.*;
import java.util.ArrayList;
import java.util.List;
import modelo.Cliente;

/**
 *
 * @author ubaldo
 */
public class ClienteDAO {
    //CRUD - Create - Read - Update - Delete
    private static final String SQL_SELECT =
```

```
        "SELECT id_cliente, nombre, apellido, email, telefono, saldo FROM clientes";
    private static final String SQL_INSERT =
        "INSERT INTO clientes(nombre, apellido, email, telefono, saldo) VALUES(?, ?,
?, ?, ?)";
    private static final String SQL_SELECT_BY_ID =
        "SELECT id_cliente, nombre, apellido, email, telefono, saldo FROM clientes
WHERE id_cliente = ?";
    private static final String SQL_UPDATE =
        "UPDATE clientes SET nombre=?, apellido=?, email=?, telefono=?, saldo=? WHERE
id_cliente=?";
    private static final String SQL_DELETE =
        "DELETE FROM clientes WHERE id_cliente = ?";

    public List<Cliente> listar(){
        List<Cliente> clientes = new ArrayList<>();
        try(Connection conn = Conexion.getConnection();
            PreparedStatement stmt = conn.prepareStatement(SQL_SELECT);
            ResultSet rs = stmt.executeQuery()){
            // Iteramos los clientes de bd
            while(rs.next()){
                Cliente cliente = new Cliente(
                    rs.getInt("id_cliente"),
                    rs.getString("nombre"),
                    rs.getString("apellido"),
                    rs.getString("email"),
                    rs.getString("telefono"),
                    rs.getDouble("saldo")
                );
                clientes.add(cliente);
            }
        } catch (SQLException ex) {
            ex.printStackTrace(System.out);
        }
        return clientes;
    }

    public int insertar(Cliente cliente){
        int rows = 0;
        try(Connection conn = Conexion.getConnection();
            PreparedStatement stmt = conn.prepareStatement(SQL_INSERT)){
            stmt.setString(1, cliente.getNombre());
            stmt.setString(2, cliente.getApellido());
            stmt.setString(3, cliente.getEmail());
            stmt.setString(4, cliente.getTelefono());
```

```
        stmt.setDouble(5, cliente.getSaldo());

        rows = stmt.executeUpdate();
    } catch (SQLException ex) {
        ex.printStackTrace(System.out);
    }
    return rows;
}

public Cliente encontrarCliente(Cliente cliente){
    try(Connection conn = Conexion.getConnection()){
        PreparedStatement stmt = conn.prepareStatement(SQL_SELECT_BY_ID){
            stmt.setInt(1, cliente.getIdCliente());
            try(ResultSet rs = stmt.executeQuery()){
                if(rs.next()){
                    cliente.setNombre(rs.getString("nombre"));
                    cliente.setApellido(rs.getString("apellido"));
                    cliente.setEmail(rs.getString("email"));
                    cliente.setTelefono(rs.getString("telefono"));
                    cliente.setSaldo(rs.getDouble("saldo"));
                }
            }
        } catch (SQLException ex) {
            ex.printStackTrace(System.out);
        }
        return cliente;
    }
}

public int actualizar(Cliente cliente){
    int rows = 0;
    try(Connection conn = Conexion.getConnection()){
        PreparedStatement stmt = conn.prepareStatement(SQL_UPDATE){
            stmt.setString(1, cliente.getNombre());
            stmt.setString(2, cliente.getApellido());
            stmt.setString(3, cliente.getEmail());
            stmt.setString(4, cliente.getTelefono());
            stmt.setDouble(5, cliente.getSaldo());
            stmt.setInt(6, cliente.getIdCliente());

            rows = stmt.executeUpdate();
        } catch (SQLException ex) {
            ex.printStackTrace(System.out);
        }
        return rows;
    }
```

```

    }

    public int eliminar(Cliente cliente){
        int rows = 0;
        try(Connection conn = Conexion.getConnection()){
            PreparedStatement stmt = conn.prepareStatement(SQL_DELETE)){

                stmt.setInt(1, cliente.getIdCliente());

                rows = stmt.executeUpdate();
            } catch (SQLException ex) {
                ex.printStackTrace(System.out);
            }
            return rows;
        }
    }
}

```

Explicación de la Implementación

1. **listar()**: Devuelve una lista de todos los clientes en la base de datos. Utiliza un try-with-resources para asegurarse de que el ResultSet, PreparedStatement y Connection se cierran automáticamente después de su uso.
2. **encontrar(Cliente cliente)**: Busca un cliente específico por su ID. Si el cliente es encontrado, se llenan los datos del objeto Cliente proporcionado.
3. **insertar(Cliente cliente)**: Inserta un nuevo cliente en la base de datos usando los datos del objeto Cliente. Retorna el número de filas afectadas, útil para saber si la operación fue exitosa.
4. **actualizar(Cliente cliente)**: Actualiza los datos de un cliente existente. Se usa el ID del cliente para localizar el registro en la base de datos y actualizar los campos correspondientes.
5. **eliminar(Cliente cliente)**: Elimina un cliente basado en su ID.

Creación del RootServlet para redirigir al ServletControlador

Este servlet raíz (root) será el punto de entrada principal de la aplicación. Este Servlet redirige automáticamente al ServletControlador usando la anotación `@WebServlet("/")`. Esto interceptará todas las solicitudes a la raíz de la aplicación y redirigirá al servlet controlador, esto es similar a un archivo de bienvenida.

```

package web;

import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;

```

```
import jakarta.servlet.http.*;

@WebServlet(name = "RootServlet", urlPatterns = {"/"})
public class RootServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.sendRedirect(request.getContextPath() + "/ServletControlador");
    }
}
```

Creación de Servlet Controlador (ServletControlador)

1. **Descripción:** Este servlet controla las acciones de mostrar, editar, insertar y eliminar clientes.
2. **Métodos Implementados:**
 - **doGet:** Determina la acción a realizar según el parámetro recibido (editar, eliminar, o default).
 - **doPost:** Inserta o actualiza registros de clientes según el formulario enviado.

```
package web;

import datos.ClienteDAO;
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import java.util.List;
import java.util.Optional;
import modelo.Cliente;

/**
 *
 * @author ubaldo
 */
@WebServlet(name = "ServletControlador", urlPatterns = {"/ServletControlador"})
public class ServletControlador extends HttpServlet {

    /**
     * Handles the HTTP <code>GET</code> method.
     */
}
```

```
*
* @param request servlet request
* @param response servlet response
* @throws ServletException if a servlet-specific error occurs
* @throws IOException if an I/O error occurs
*/
@Override
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    String accion =
Optional.ofNullable(request.getParameter("accion")).orElse("listar");
    switch(accion){
        case "listar" -> this.listarClientes(request, response);
        case "editar" -> this.editarCliente(request, response);
        case "eliminar" -> this.eliminarCliente(request, response);
        default -> this.listarClientes(request, response);
    }
}

private void listarClientes(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException{
    List<Cliente> clientes = new ClienteDAO().listar();
    System.out.println("clientes:" + clientes);
    // Obtenemos la sesion
    HttpSession session = request.getSession();
    session.setAttribute("clientes", clientes);
    session.setAttribute("totalClientes", clientes.size());
    session.setAttribute("saldoTotal", this.calcularSaldoTotal(clientes));
    // Enviamos la respuesta al jsp de clientes
    request.getRequestDispatcher("clientes.jsp").forward(request, response);
}

private double calcularSaldoTotal(List<Cliente> clientes){
//    double saldoTotal = 0.0;
//    for(Cliente cliente: clientes){
//        saldoTotal += cliente.getSaldo();
//    }
//    return saldoTotal;

    return clientes.stream().mapToDouble(Cliente::getSaldo).sum();
}

private void editarCliente(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException{
    int idCliente = Integer.parseInt(request.getParameter("idCliente"));
}
```

```

        Cliente cliente = new ClienteDAO().encontrarCliente(new Cliente(idCliente));
        request.setAttribute("cliente", cliente);
        String jspEditar = "/WEB-INF/paginas/cliente/editarCliente.jsp";
        request.getRequestDispatcher(jspEditar).forward(request, response);
    }

    private void eliminarCliente(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException{
        int idCliente = Integer.parseInt(request.getParameter("idCliente"));
        new ClienteDAO().eliminar(new Cliente(idCliente));
        // Listamos los clientes
        this.listarClientes(request, response);
    }

    /**
     * Handles the HTTP <code>POST</code> method.
     *
     * @param request servlet request
     * @param response servlet response
     * @throws ServletException if a servlet-specific error occurs
     * @throws IOException if an I/O error occurs
     */
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        String accion =
Optional.ofNullable(request.getParameter("accion")).orElse("listar");
        switch(accion){
            case "insertar" -> this.insertarCliente(request, response);
            case "modificar" -> this.modificarCliente(request, response);
            default -> this.listarClientes(request, response);
        }
    }

    private void insertarCliente(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {
        // Procesamos los datos del formulario
        String nombre = request.getParameter("nombre");
        String apellido = request.getParameter("apellido");
        String email = request.getParameter("email");
        String telefono = request.getParameter("telefono");
        double saldo = Double.parseDouble(request.getParameter("saldo"));

```

```

        // Creamos un objeto de tipo cliente
        Cliente cliente = new Cliente(nombre, apellido, email, telefono, saldo);
        new ClienteDAO().insertar(cliente);
        // Listamos los clientes
        this.listarClientes(request, response);
    }

    private void modificarCliente(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {
        int idCliente = Integer.parseInt(request.getParameter("idCliente"));
        String nombre = request.getParameter("nombre");
        String apellido = request.getParameter("apellido");
        String email = request.getParameter("email");
        String telefono = request.getParameter("telefono");
        double saldo = Double.parseDouble(request.getParameter("saldo"));

        Cliente cliente = new Cliente(idCliente, nombre, apellido, email, telefono,
saldo);
        new ClienteDAO().actualizar(cliente);
        // Redirigimos al caso de listar
        this.listarClientes(request, response);
    }
}

```

Explicación del método calcularSaldoTotal():

En Java, un **Stream** es una secuencia de elementos que permite realizar operaciones funcionales y de procesamiento de datos de manera concisa y eficiente. Los Streams fueron introducidos en **Java 8** como parte de la API de `java.util.stream` y están diseñados para trabajar con colecciones de datos, como listas o conjuntos, de una manera declarativa y funcional.

Vamos a desglosar este código paso a paso:

```
return clientes.stream().mapToDouble(Cliente::getSaldo).sum();
```

Este código utiliza la API de Streams de Java para calcular la suma de los saldos de una lista de objetos `Cliente`. La lista `clientes` es de tipo `List<Cliente>`, y cada `Cliente` tiene un atributo `saldo` que representa su saldo.

Aquí está el paso a paso:

1. `clientes.stream()`

- **Descripción:** Convierte la lista `clientes` en un `Stream` de objetos `Cliente`.
- **Función de `stream()`:** Este método permite tratar cada elemento de la lista `clientes` de forma secuencial, y realizar operaciones en cada uno de ellos sin modificar la lista original.
- **Resultado:** Al final de esta llamada, tenemos un `Stream<Cliente>`, que es una secuencia de elementos `Cliente` que puede ser procesada en cadena.

2. `.mapToDouble(Cliente::getSaldo)`

- **Descripción:** La operación `mapToDouble` transforma cada elemento del `Stream<Cliente>` en un valor `double` correspondiente al saldo de cada `Cliente`.
- **`Cliente::getSaldo`:** Esta es una referencia a un método, que llama el método `getSaldo()` en cada objeto `Cliente` del `Stream`. Básicamente, por cada `Cliente` en el flujo, se invoca `getSaldo()`, y el valor retornado (de tipo `double`) es el saldo de ese cliente.
- **Resultado:** El `Stream<Cliente>` se convierte en un `DoubleStream`, que es un flujo de elementos `double`. Este flujo de valores representa los saldos de todos los clientes en la lista.

3. `.sum()`

- **Descripción:** La operación `sum()` toma el `DoubleStream` y suma todos los elementos en el flujo, es decir, todos los saldos de los clientes.
- **Resultado:** Devuelve un valor `double` que representa la suma total de los saldos de todos los `Cliente` en la lista `clientes`.

Ejemplo Paso a Paso con Datos

Supongamos que tenemos la lista de clientes con los siguientes saldos:

- Cliente 1: saldo 100.0
- Cliente 2: saldo 150.5
- Cliente 3: saldo 200.0

El flujo de operaciones sería:

1. `clientes.stream()` crea un flujo de los tres `Cliente`.
2. `.mapToDouble(Cliente::getSaldo)` transforma el flujo a un `DoubleStream` con los valores 100.0, 150.5, 200.0.
3. `.sum()` suma estos valores: $100.0 + 150.5 + 200.0 = 450.5$.

Entonces, el código devuelve 450.5 como la suma del total de los saldos.

Páginas JSP y Frontend

Las páginas JSP (`index.jsp`, `clientes.jsp`, `agregarCliente.jsp`, `editarCliente.jsp`) utilizan Bootstrap para un diseño moderno y responsivo. Se han aplicado las siguientes prácticas:

1. **Uso de Iconos de Bootstrap:** Se han utilizado iconos de Bootstrap para mantener la consistencia con el uso de Bootstrap.
2. **Validación de Formularios:** Se han añadido clases para validación, como `was-validated`, en formularios.

Archivo clientes.jsp

Descripción: Página principal que muestra el listado de clientes con botones para agregar, editar y eliminar registros.

```
<!doctype html>
<html data-bs-theme="dark">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Control de Clientes</title>
    <!--Bootstrap CSS-->
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhY6hW+ALEwIH"
crossorigin="anonymous">
    <!--Bootstrap Iconos-->
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.11.3/font/bootstrap-icons.min.css">
  </head>
  <body>
    <!--Cabecero-->
    <jsp:include page="/WEB-INF/paginas/comunes/cabecero.jsp"/>

    <!--Botones de Navegación-->
    <jsp:include page="/WEB-INF/paginas/comunes/botonesNavegacion.jsp"/>

    <!--Listado de Clientes-->
    <jsp:include page="/WEB-INF/paginas/cliente/listadoClientes.jsp"/>

    <!--Modal agregar un cliente-->
    <jsp:include page="/WEB-INF/paginas/cliente/agregarCliente.jsp"/>

    <!--Pie página-->
    <jsp:include page="/WEB-INF/paginas/comunes/piePagina.jsp"/>
```

```

    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDz0xhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>
  </body>
</html>

```

Este código HTML utiliza Bootstrap para aplicar estilos y funcionalidades de diseño en la aplicación de "Control de Clientes". Además de agregar los links necesarios para trabajar con Bootstrap y los íconos de Bootstrap se realizaron las siguientes configuraciones básicas:

1. **<html data-bs-theme="dark">**: Activa el tema oscuro de Bootstrap en todo el documento.
2. **Meta etiquetas (<meta>)**:
 - o **viewport**: Configura la vista para dispositivos móviles, ajustando el contenido a la pantalla.

Dentro de WEB-INF, creamos las carpetas páginas y después las carpetas de cliente y comunes para agregar los JSP de nuestra aplicación.

Archivo WEB-INF/paginas/comunes/cabecero.jsp

```

<header id="main-header" class="py-2 bg-info text-white">
  <div class="container">
    <div class="d-flex align-items-center">
      <h1><i class="bi bi-people-fill"></i> Control de Clientes</h1>
    </div>
  </div>
</header>

```

Aquí tienes una explicación de cada clase utilizada en el código anterior:

1. **py-2**: Padding Y-axis 2. Aplica relleno vertical (arriba y abajo) de `0.5rem`, creando espacio adicional en el eje vertical.
2. **bg-info**: Background Info. Asigna un fondo color azul claro, que suele usarse para indicar información relevante.
3. **text-white**: Texto Blanco. Cambia el color del texto a blanco para contrastar bien sobre fondos oscuros o de color.
4. **container**: Contenedor. Limita el ancho del contenido y lo centra horizontalmente, creando márgenes laterales automáticos.
5. **d-flex**: Display Flex. Activa flexbox, permitiendo alinear y distribuir elementos secundarios de manera flexible.

6. **align-items-center**: Alinear Elementos al Centro. Centra verticalmente los elementos secundarios en el contenedor de flexbox.
7. **bi bi-people-fill**: Bootstrap Icons - Ícono de Personas. **bi** activa los iconos de Bootstrap, y **bi-people-fill** muestra un icono de "personas" relleno, útil para representar usuarios o clientes.

Archivo WEB-INF/paginas/comunes/botonesNavegacion.jsp

```
<section id="actions" class="py-4 mb-4">
  <div class="container">
    <div class="row">
      <div class="col-md-3">
        <button type="button" class="btn btn-primary btn-block w-100"
          data-bs-toggle="modal" data-bs-target="#agregarClienteModal">
          <i class="bi bi-plus-circle"></i> Agregar Cliente
        </button>
      </div>
    </div>
  </div>
</section>
```

Aquí tienes una explicación de las clases utilizadas en el código anterior:

1. **py-4**: Padding Y-axis 4. Aplica relleno vertical de `1.5rem`, añadiendo espacio arriba y abajo.
2. **mb-4**: Margin Bottom 4. Aplica un margen inferior de `1.5rem`, creando espacio entre secciones.
3. **container**: Contenedor. Centra y limita el ancho del contenido en la página.
4. **row**: Fila. Activa el sistema de cuadrícula para colocar las columnas.
5. **col-md-3**: Columna de 3/12 para pantallas medianas en adelante (equivale al 25% del ancho).
6. **btn**: Botón. Estilo base para botones en Bootstrap.
7. **btn-primary**: Botón Primario. Cambia el color del botón al color principal de Bootstrap (azul por defecto).
8. **btn-block** (Bootstrap 4) y **w-100** (Bootstrap 5): Botón de ancho completo. Hace que el botón ocupe el 100% del ancho del contenedor.
9. **data-bs-toggle="modal"**: Muestra el modal al hacer clic en el botón.
10. **data-bs-target="#agregarClienteModal"**: Especifica el `id` del modal a abrir.
11. **bi bi-plus-circle**: Bootstrap Icon. Muestra un icono de círculo con un símbolo de "+" para indicar la acción de agregar.

Archivo WEB-INF/paginas/comunes/piePagina.jsp

```
<footer id="pie-pagina" class="bg-warning text-white mt-5 p-5">
  <div class="container">
    <p class="lead text-center">© GlobalMentoring.com.mx</p>
  </div>
</footer>
```

Aquí tienes la descripción de las clases utilizadas en el código anterior:

1. **bg-warning**: Background Warning. Aplica un fondo color amarillo de alerta.
2. **text-white**: Texto Blanco. Cambia el color del texto a blanco para buen contraste sobre fondos oscuros o coloridos.
3. **mt-5**: Margin Top 5. Aplica un margen superior de `3rem`, separando el pie de página del contenido anterior.
4. **p-5**: Padding 5. Aplica relleno de `3rem` en todos los lados del contenedor, creando espacio interior.
5. **container**: Contenedor. Centra el contenido y limita su ancho.
6. **lead**: Texto Lead. Aumenta ligeramente el tamaño y el peso del texto para destacar el estilo.
7. **text-center**: Texto Centrado. Centra horizontalmente el contenido dentro del elemento.

Archivo WEB-INF/paginas/cliente/listadoClientes.jsp

```
<%@taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
<!--Listado Clientes-->
<section id="clientes">
  <div class="container">
    <div class="row">
      <div class="col-md-9">
        <div class="card">
          <div class="card-header">
            <h4>Listado de Clientes</h4>
          </div>
          <table class="table table-striped">
            <thead class="thead-dark">
              <tr>
                <th>#</th>
                <th>Nombre</th>
                <th>Saldo</th>
                <th>Acciones</th>
              </tr>
            </thead>
          </table>
        </div>
      </div>
    </div>
  </div>
```

```

        </tr>
    </thead>
    <tbody>
        <c:forEach var="cliente" items="${clientes}"
varStatus="status">
            <tr>
                <td>${status.count}</td>
                <td>${cliente.nombre} ${cliente.apellido}</td>
                <td><fmt:formatNumber value="${cliente.saldo}"
                    type="currency"/>
                </td>
                <td>
                    <a
href="${pageContext.request.contextPath}/ServletControlador?accion=editar&idCliente=${cli
ente.idCliente}"
                    class="btn btn-secondary btn-sm">
                        <i class="bi bi-pencil"></i> Editar
                    </a>
                    <a
href="${pageContext.request.contextPath}/ServletControlador?accion=eliminar&idCliente=${c
liente.idCliente}"
                    class="btn btn-danger btn-sm">
                        <i class="bi bi-trash"></i> Eliminar
                    </a>
                </td>
            </tr>
        </c:forEach>
    </tbody>
</table>
</div>
</div>
<!--Totales-->
<div class="col-md-3">
    <div class="card text-center bg-danger text-white mb-3">
        <div class="card-body">
            <h3>Saldo Total</h3>
            <h4 class="display-6">
                <fmt:formatNumber value="${saldoTotal}" type="currency"/>
            </h4>
        </div>
    </div>
</div>

    <div class="card text-center bg-success text-white mb-3">
        <div class="card-body">
            <h3>Total Clientes</h3>

```

```

        <h4 class="display-6">
            <i class="bi bi-people"></i> ${totalClientes}
        </h4>
    </div>
</div>

</div>
</div>
</section>

```

Aquí tienes la descripción de las clases y etiquetas JSTL utilizadas:

1. `<%@taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>`: Importa las etiquetas core de JSTL, como `<c:forEach>` para iterar listas.
2. `<%@taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>`: Importa etiquetas de formato de JSTL, como `<fmt:formatNumber>` para dar formato a números y moneda.

Clases de Bootstrap

3. **container**: Centra y limita el ancho del contenido.
4. **row**: Fila de Bootstrap. Activa el sistema de cuadrícula.
5. **col-md-9** y **col-md-3**: Columnas que ocupan 9/12 y 3/12 del ancho en pantallas medianas en adelante.
6. **card**: Tarjeta. Contenedor con bordes y sombras.
7. **card-header**: Cabecera de Tarjeta. Define el área de encabezado.
8. **table**: Estilo de tabla con bordes y espaciado.
9. **table-striped**: Aplica fondo alternado a las filas de la tabla.
10. **thead-dark**: Cabecera de tabla en fondo oscuro y texto claro.
11. **btn**: Botón básico.
12. **btn-secondary** y **btn-danger**: Botón en color secundario (gris) y color de peligro (rojo).
13. **btn-sm**: Botón pequeño.
14. **bi bi-pencil** y **bi bi-trash**: Íconos de lápiz y papelera de Bootstrap.
15. **text-center**: Centra el texto horizontalmente.
16. **bg-danger** y **bg-success**: Fondos de color rojo (peligro) y verde (éxito).
17. **text-white**: Texto blanco.
18. **mb-3**: Margen inferior de 1rem.
19. **card-body**: Cuerpo de tarjeta. Contenedor interno del contenido de la tarjeta.
20. **display-6**: Tamaño de encabezado grande (1.25rem), ideal para resaltar números y totales.

Archivo WEB-INF/paginas/cliente/agregarCliente.jsp

```

<div class="modal fade" id="agregarClienteModal" tabindex="-1" aria-
labelledby="agregarClienteLabel"
    aria-hidden="true">
    <div class="modal-dialog modal-lg">
        <div class="modal-content">
            <div class="modal-header bg-info text-white">
                <h5 class="modal-title" id="agregarClienteLabel">Agregar Cliente</h5>
                <button type="button" class="btn-close" data-bs-dismiss="modal" aria-
label="Close"></button>
            </div>
            <!--Formulario para agregar un nuevo cliente-->
            <form
action="${pageContext.request.contextPath}/ServletControlador?accion=insertar"
    method="POST" class="was-validated">
                <div class="modal-body">
                    <div class="form-group mb-3">
                        <label for="nombre">Nombre</label>
                        <input type="text" class="form-control" id="nombre" name="nombre"
required/>
                    </div>
                    <div class="form-group mb-3">
                        <label for="apellido">Apellido</label>
                        <input type="text" class="form-control" id="apellido"
name="apellido" required/>
                    </div>
                    <div class="form-group mb-3">
                        <label for="email">Email</label>
                        <input type="email" class="form-control" id="email" name="email"
required />
                    </div>
                    <div class="form-group mb-3">
                        <label for="telefono">Teléfono</label>
                        <input type="tel" class="form-control" id="telefono"
name="telefono" required />
                    </div>
                    <div class="form-group mb-3">
                        <label for="saldo">Saldo</label>
                        <input type="number" class="form-control" id="saldo" name="saldo"
required
                            step="any"/>
                    </div>
                </div>
            <!--Pie página de la ventana modal-->

```



```

        <div class="modal-footer">
            <button type="button" class="btn btn-secondary" data-bs-
dismiss="modal">Cancelar</button>
            <button type="submit" class="btn btn-primary">Guardar</button>
        </div>
    </form>
</div>
</div>
</div>

```

Aquí tienes la explicación de las clases y atributos Bootstrap utilizados en el código anterior:

1. **modal fade**: Modal Fade. Crea una ventana modal con efecto de desvanecimiento.
2. **id="agregarClienteModal"**: Asigna un identificador al modal para referenciarlo en el código JavaScript o HTML.
3. **tabindex="-1"**: Remueve el foco del modal, para no interferir con el enfoque de otros elementos.
4. **aria-labelledby** y **aria-hidden**: Accesibilidad. **aria-labelledby** asocia el título del modal y **aria-hidden** oculta el modal a lectores de pantalla cuando no está visible.

Clases de Bootstrap en Elementos Internos

5. **modal-dialog**: Diálogo de Modal. Define el área interna de la ventana modal.
6. **modal-lg**: Modal Large. Establece un ancho grande para el modal.
7. **modal-content**: Contenido de Modal. Define el área de contenido del modal con estilos específicos.
8. **modal-header**: Cabecera de Modal. Contiene el encabezado y estilo del título del modal.
9. **bg-info** y **text-white**: Fondo informativo (azul claro) y texto blanco para el encabezado.
10. **modal-title**: Título del Modal. Estilo específico para el encabezado del modal.
11. **btn-close**: Botón de Cierre. Botón pequeño para cerrar el modal.
12. **data-bs-dismiss="modal"**: Cierra el modal al hacer clic en el botón.

Clases del Formulario

13. **was-validated**: Activación de validación de Bootstrap en el formulario.
14. **form-group**: Grupo de Formulario. Agrupa y organiza los campos del formulario.
15. **mb-3**: Margin Bottom 3. Margen inferior de `1rem` para cada campo del formulario.
16. **form-control**: Control de Formulario. Proporciona estilos y espaciado para los campos de entrada.

Footer del Modal

17. **modal-footer**: Pie del Modal. Área inferior del modal para botones de acción.
18. **btn btn-secondary**: Botón gris para "Cancelar".
19. **btn btn-primary**: Botón azul principal para "Guardar".

Estas clases crean un modal interactivo, organizado y accesible, facilitando la entrada de datos en la interfaz.

Archivo WEB-INF/paginas/comunes/botonesNavegacionEdicion.jsp

```
<section id="actions" class="py-4 mb-4">
  <div class="container">
    <div class="row">
      <div class="col-md-3">
        <a href="${pageContext.request.contextPath}/"
          class="btn btn-secondary btn-block w-100">
          <i class="bi bi-arrow-left "></i> Regresar al inicio
        </a>
      </div>
      <div class="col-md-3">
        <button type="submit" class="btn btn-success btn-block w-100">
          <i class="bi bi-check-lg"></i> Guardar Cliente
        </button>
      </div>
      <div class="col-md-3">
        <a
href="${pageContext.request.contextPath}/ServletControlador?accion=eliminar&idCliente=${c
liente.idCliente}"
          class="btn btn-danger btn-block w-100">
          <i class="bi bi-trash"></i> Eliminar Cliente
        </a>
      </div>
    </div>
  </div>
</section>
```

Aquí tienes la explicación detallada y compacta de cada clase utilizada en el código anterior:

1. **py-4**: Padding Y-axis 4. Añade un relleno vertical de `1.5rem`, creando espacio arriba y abajo.

2. **mb-4**: Margin Bottom 4. Añade un margen inferior de `1.5rem` para separar esta sección del contenido siguiente.

Clases de Bootstrap en Elementos Internos

3. **container**: Contenedor. Centra y limita el ancho del contenido.
4. **row**: Fila. Activa el sistema de cuadrícula de Bootstrap.
5. **col-md-3**: Columna de 3/12 en pantallas medianas en adelante, ocupando un 25% del ancho.

Clases de los Botones

6. **btn**: Botón básico. Aplica estilos de botón predeterminados de Bootstrap.
7. **btn-secondary**: Botón Secundario (gris), usado para el enlace de "Regresar al inicio".
8. **btn-success**: Botón de Éxito (verde), usado para el botón "Guardar Cliente".
9. **btn-danger**: Botón de Peligro (rojo), usado para el enlace "Eliminar Cliente".
10. **btn-block** (Bootstrap 4) y **w-100** (Bootstrap 5): Botón de ancho completo, ocupa el 100% del ancho del contenedor.

Iconos de Bootstrap

11. **bi bi-arrow-left**: Icono de flecha hacia la izquierda para indicar "Regresar al inicio".
12. **bi bi-check-lg**: Icono de marca de verificación grande para indicar "Guardar".
13. **bi bi-trash**: Icono de papelera para indicar "Eliminar".

Este conjunto de clases organiza y estiliza una sección de acciones, presentando botones con un diseño uniforme, icónico y claramente diferenciado según la acción que representan.

Archivo WEB-INF/paginas/cliente/editarCliente.jsp (se vuelve a agregar Bootstrap)

```
<!doctype html>
<html data-bs-theme="dark">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Editar Cliente</title>
    <!--Bootstrap CSS-->
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
```

```

QWTKZyjpPEjISv5WaRU90FeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
    <!--Bootstrap Iconos-->
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-
icons@1.11.3/font/bootstrap-icons.min.css">
</head>
<body>
    <!--Cabecero-->
    <jsp:include page="/WEB-INF/paginas/comunes/cabecero.jsp"/>

    <!--Formulario de edicion de un cliente-->
    <form
action="${pageContext.request.contextPath}/ServletControlador?accion=modificar&idCliente=
${cliente.idCliente}"
        method="POST" class="was-validated">

        <!--Botones de navegacion de edicion-->
        <jsp:include page="/WEB-INF/paginas/comunes/botonesNavegacionEdicion.jsp"/>

        <div class="container my-4">
            <div class="card">
                <div class="card-header bg-primary text-white">
                    <h4>Editar Cliente</h4>
                </div>
                <div class="card-body">

                    <div class="form-group mb-3">
                        <label for="nombre">Nombre</label>
                        <input type="text" class="form-control" id="nombre"
name="nombre" required
                            value="${cliente.nombre}"/>
                    </div>
                    <div class="form-group mb-3">
                        <label for="apellido">Apellido</label>
                        <input type="text" class="form-control" id="apellido"
name="apellido" required
                            value="${cliente.apellido}"/>
                    </div>
                    <div class="form-group mb-3">
                        <label for="email">Email</label>
                        <input type="email" class="form-control" id="email"
name="email" required
                            value="${cliente.email}"/>
                    </div>
                    <div class="form-group mb-3">

```

```

        <label for="telefono">Teléfono</label>
        <input type="tel" class="form-control" id="telefono"
name="telefono" required
        value="${cliente.telefono}"/>
    </div>
    <div class="form-group mb-3">
        <label for="saldo">Saldo</label>
        <input type="number" class="form-control" id="saldo"
name="saldo" required
        step="any" value="${cliente.saldo}"/>
    </div>
    <button type="submit" class="btn btn-success">Guardar
Cambios</button>

    </div>
</div>

</div>

</form>

<!--Pie página-->
<jsp:include page="/WEB-INF/paginas/comunes/piePagina.jsp"/>

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
integrity="sha384-YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDz0xhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>
    </body>
</html>

```

Aquí tienes la explicación de cada clase y configuración utilizada en el código anterior:

Configuración Inicial de la Página

1. `<!doctype html>`: Define el documento como HTML5.
2. `<html data-bs-theme="dark">`: Activa el tema oscuro de Bootstrap.
3. `<meta charset="utf-8">`: Define el conjunto de caracteres como UTF-8.
4. `<meta name="viewport" content="width=device-width, initial-scale=1">`: Ajusta el diseño a dispositivos móviles.
5. `<link> Bootstrap CSS`: Incluye el archivo CSS de Bootstrap.
6. `<link> Bootstrap Icons`: Incluye los íconos de Bootstrap para su uso en el proyecto.

Clases de Bootstrap en el Contenido Principal

7. **container my-4:**
 - **container:** Centra el contenido y limita el ancho.
 - **my-4:** Margín Y-axis 4. Añade un margen vertical de `1.5rem` (arriba y abajo).

Clases del Formulario y Campos

8. **was-validated:** Habilita estilos de validación de Bootstrap en el formulario.
9. **card:** Crea una tarjeta con bordes y sombra.
10. **card-header bg-primary text-white:**
 - **card-header:** Cabecera de la tarjeta.
 - **bg-primary:** Fondo azul para resaltar el encabezado.
 - **text-white:** Cambia el color del texto a blanco para contraste.
11. **form-group mb-3:**
 - **form-group:** Agrupa cada campo del formulario.
 - **mb-3:** Margín Bottom 3. Añade un margen inferior de `1rem` entre campos.
12. **form-control:** Aplica estilos Bootstrap para los campos de entrada (`input`), haciendo que luzcan consistentes.

Botones

13. **btn btn-success:**
 - **btn:** Botón básico de Bootstrap.
 - **btn-success:** Botón verde, usado para acciones de éxito como "Guardar Cambios".

JavaScript de Bootstrap

14. **<script> Bootstrap JS:** Incluye el archivo JavaScript de Bootstrap, activando funcionalidades como modales y botones.

Este conjunto de clases de Bootstrap permite construir un formulario de edición de cliente visualmente agradable, organizado y responsivo, adecuado para interfaces de usuario modernas.

Al finalizar veremos nuestra aplicación de Control de Clientes

Listado de Clientes:

The screenshot shows a web browser window with the address bar displaying 'localhost:8080/ControlClientes/ServletControlador'. The page title is 'Control de Clientes'. At the top, there is a blue button labeled '⊕ Agregar Cliente'. Below this is a table titled 'Listado de Clientes' with the following data:

#	Nombre	Saldo	Acciones
1	Renata Cruz	\$100.00	✎ Editar 🗑 Eliminar
2	Victoria Reyes	\$80.00	✎ Editar 🗑 Eliminar
3	Diego Flores	\$70.00	✎ Editar 🗑 Eliminar

To the right of the table, there are two summary boxes: a red one for 'Saldo Total \$250.00' and a green one for 'Total Clientes 3' (with a person icon). The footer of the page is a yellow bar with the text '© GlobalMentoring.com.mx'.

Agregar un nuevo Cliente:

The screenshot shows a web browser window with the address bar displaying 'localhost:8080/ControlClientes/ServletControlador'. The page title is 'Control de Clientes'. A modal dialog titled 'Agregar Cliente' is open, containing the following fields:

- Nombre: Ubaldo (with a green checkmark)
- Apellido: Acosta (with a green checkmark)
- Email: ubaldoacosta@gmail.com
- Teléfono: 5500998877 (with a red '...' icon and a green checkmark)
- Saldo: 400 (with a green checkmark)

At the bottom right of the modal are two buttons: 'Cancelar' and 'Guardar'.

Editar un Cliente ya existente:

The screenshot displays a web browser window with the address bar showing `localhost:8080/ControlClientes/ServletControlador?accion=...`. The page title is "Control de Clientes". Below the title, there are three buttons: "← Regresar al inicio", "✓ Guardar Cliente", and "🗑 Eliminar Cliente". The main content area is titled "Editar Cliente" and contains a form with the following fields:

- Nombre: ✓
- Apellido: ✓
- Email: ✓
- Teléfono: ✓
- Saldo: ✓

At the bottom of the form is a button labeled "Guardar Cambios". The footer of the page is a yellow bar with the text "© GlobalMentoring.com.mx".

Con este proyecto concluimos la sección de Servlets y JPSs.

Muchas felicidades por haber concluido esta sección, ahora tienes mucho conocimiento para desarrollar tus propias aplicaciones Web trabajando con Servlets/JSPs, HTML, CSS, Bootstrap y MySql entre varias tecnologías más.

Nos vemos en más lecciones de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)